Despliegue card-less en AWS

Cómo se desplegó Neptuno en AWS **sin tarjeta de crédito y con costo de USD $0**, y por qué eso no es una limitación sino la garantía del propio plan.

> Documento de la ejecución real, no de un supuesto. Cada restricción de la sección 2 se comprobó contra la API; las que se dieron por supuestas resultaron falsas y están anotadas como tal.

---

1. Qué significa “card-less” aquí

AWS ofrece un **plan gratuito que no exige tarjeta**. A cambio, la cuenta no puede gastar: los recursos fuera del plan se rechazan de plano en vez de facturarse.

La cuenta del proyecto:

Dato	Valor
Plan	Gratuito (sin tarjeta)
Créditos	USD $100
Vigencia	183 días
Gasto acumulado	**USD $0**

Esto tiene una consecuencia que conviene entender: **las restricciones no son defectos del plan, son su mecanismo de defensa.** Cuando AWS rechaza un tipo de instancia, no está fallando: está impidiendo que la factura crezca. El candado y el $0 son la misma cosa.

No activar “funciones avanzadas”

La consola ofrece pasar al plan de pago. Esa pantalla advierte que la activación **elimina los límites de gasto**. Es exactamente el candado que hoy protege el presupuesto: activarlo lo borra y a partir de ahí la cuenta gasta de verdad. No tocarlo.

---

2. Restricciones reales (comprobadas)

Se intentó	Resultado	Consecuencia
`t3.medium`	`InvalidParameterCombination` : *“is not eligible for Free Tier”*	Solo se admiten tipos elegibles (sección 3)
Registrar dominio	`AccessDeniedException` : *“Free Tier accounts are not supported for this service”*	**No se compra dominio**; HTTPS se resuelve con CloudFront
`account:GetContactInformation`	Denegado por política de organización (SCP)	El correo de alertas de presupuesto se indica a mano
Clave de acceso IAM previa	`InvalidClientTokenId`	Se descarta y se usa `aws login` (sección 4)

Los dos primeros son bloqueos duros: no hay ajuste que los habilite.

---

3. Lo que sí entra

Tipos de instancia elegibles en `us-east-2`:

Tipo	vCPU	RAM
`m7i-flex.large`	2	**8 GB**
`c7i-flex.large`	2	8 GB
`t3.small`	2	2 GB
`t3.micro`	2	1 GB

Se eligió **`m7i-flex.large`**: 8 GB cubren de sobra los ~6,2 GB que necesitaría el proyecto completo (4 microservicios + MinIO), y es `x86_64`, así que no hay que recompilar nada para ARM. `t3.small` (2 GB) no alcanza para el stack.

En la ejecución documentada, el uso real fue de **1,9 GB de 7,8 GB**.

---

4. Autenticación sin claves estáticas

No se usan claves de acceso. El CLI se autentica contra la sesión de la consola con credenciales temporales que se renuevan solas:

```
aws configure set region us-east-2 --profile <perfil>
aws login --profile <perfil>          # abre el navegador; no hay claves que copiar
aws sts get-caller-identity --profile <perfil>
```

Ventajas: no hay secretos de larga vida en disco y nada que rotar. `aws login` debe ejecutarse en una terminal real (necesita TTY); en un host remoto se usa `--remote`.

---

5. Despliegue paso a paso

5.1 Red y acceso

```
P="--profile <perfil> --region us-east-2"

aws $P ec2 create-key-pair --key-name neptuno --key-type rsa --key-format pem \
  --query 'KeyMaterial' --output text > ~/neptuno.pem
chmod 400 ~/neptuno.pem

aws $P ec2 create-security-group --group-name neptuno-sg \
  --description "Neptuno - SSH admin y frontend 8080" --query 'GroupId' --output text

# Solo 22 desde la IP propia, y 8080 público (es la entrada del sistema).
```

```
        aws $P ec2 authorize-security-group-ingress --group-id <sg> \
          --ip-permissions 'IpProtocol=tcp,FromPort=22,ToPort=22,IpRanges=[{CidrIp=<IP-propia>/
32}]' \

'IpProtocol=tcp,FromPort=8080,ToPort=8080,IpRanges=[{CidrIp=0.0.0.0/0}]'
```

Los puertos 8081, 8082, 3306 y 15672 **no se abren**: el frontend llega a los servicios por la red interna de contenedores.

5.2 Instancia

```
        aws $P ec2 run-instances --image-id ami-09519cc56eb20524a \
          --instance-type m7i-flex.large --key-name neptuno \
          --block-device-mappings 'DeviceName=/dev/
sda1,Ebs={VolumeSize=20,VolumeType=gp3,DeleteOnTermination=true}' \
          --network-interfaces
'DeviceIndex=0,SubnetId=<subred>,Groups=<sg>,AssociatePublicIpAddress=true' \
          --metadata-options 'HttpTokens=required,HttpEndpoint=enabled' \
          --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=neptuno}]'
```

`HttpTokens=required` fuerza IMDSv2, que evita el robo de credenciales por SSRF.

5.3 Elastic IP (obligatoria)

CloudFront necesita un **origen con nombre estable**. Sin IP elástica, cada Stop/Start cambia el DNS público de la instancia y el CDN responde 502.

```
        aws $P ec2 allocate-address --domain vpc
        aws $P ec2 associate-address --instance-id <id> --allocation-id <eip>
```

No añade coste: una IPv4 pública ya se factura igual, esté asociada o no.

5.4 Aplicación

```
        scp -r ./ <usuario>@<ip>:~/Neptuno/
        ssh <usuario>@<ip> 'cd ~/Neptuno && ./desplegar-ec2.sh'
```

El script instala Podman, genera `.env` con secretos aleatorios y levanta el stack. La clave del administrador inicial queda **solo** en la instancia:

```
        grep ADMIN_CLAVE ~/Neptuno/.env
```

---

6. Trampas de Podman (las tres cuestan horas si no se conocen)

Pasar de Docker a Podman en Ubuntu 24.04 rompe tres cosas. Las tres están corregidas en el repositorio:

1. **Nombres cortos de imagen** (`mysql:8.4`, `node:24-alpine`). Ubuntu deja `/etc/containers/registries.conf` **sin** `unqualified-search-registries`, así que ni el `build` ni el `pull` los resuelven. Síntoma: `did not resolve to an alias and no unqualified-search registries are defined`. El script crea el drop-in `/etc/containers/registries.conf.d/00-neptuno-dockerio.conf`.
2. **El `resolver` de nginx.** `frontend/nginx.conf.template` lo fijaba a `127.0.0.11`, que es el DNS de **Docker**; Podman usa `10.89.0.1`. Síntoma engañoso: la SPA carga (son ficheros estáticos, nginx no necesita DNS) pero **toda la API muere** con `499`/`502` y `recv() failed (111: Connection refused) while resolving`. Ahora no se fija ninguno y nginx resuelve al arrancar con el `/etc/resolv.conf` del contenedor, válido en ambos motores.
3. **Contenedores que no se recrean.** `podman-compose` 1.0.x **recompone la imagen pero no recrea el contenedor** si no cambió el hash del compose. Es la peor de las tres porque es silenciosa: `git pull && ./desplegar-ec2.sh` seguiría ejecutando el código anterior. Por eso el script usa `--force-recreate`.

---

7. HTTPS sin dominio (CloudFront)

Como el registro de dominios está bloqueado, se usa el certificado y el subdominio por defecto de CloudFront. El resultado es **HTTPS con candado válido y sin coste**.

Ajuste	Valor	Motivo
Origen	DNS público de la instancia, puerto 8080, `http-only`	nginx escucha en 8080
Comportamiento por defecto	`CachingDisabled` + `AllViewerExceptHostHeader`	la SPA y `/api/*` **nunca** se cachean
`/assets/*`	`CachingOptimized`	ficheros con hash: inmutables, 1 año de caché
Protocolo del visor	`redirect-to-https`	`http://` → `301` a `https://`
Clase de precio	`PriceClass_All`	incluye los nodos de Sudamérica

El punto crítico es el segundo: **un CDN mal configurado cachea respuestas autenticadas y sirve el token de un usuario a otro.** La API conserva su `cache-control: no-store`.

Coste: dentro de la capa gratuita (1 TB/mes de salida, 10 M de peticiones). A la escala de este proyecto, $0.

Detalle conocido: los errores 4xx/5xx heredan el `ErrorCachingMinTTL` por defecto (10 s). Ponerlo a 0 lo elimina; exige otra propagación de ~3 min.

---

8. Verificación

Prueba	Resultado
Contenedores	6/6 `Up`, las 3 bases/broker `healthy`
Migraciones (Flyway)	Aplicadas; 26 permisos y 6 roles sembrados
SPA por CDN	`200`, TLS 1.3, certificado `*.cloudfront.net`
HTTP	`301` → HTTPS
Login por el CDN	`200` con `acceso`, `refresco`, `token`, `permisos`, `roles`
Estáticos	`x-cache: Miss` → **`Hit from cloudfront`** (cachea de verdad)
API	`no-cache, no-store` (no se cachea)
Puertos 8081/8082	**cerrados** al exterior
Puerto 22	solo desde la IP del equipo

---

9. Costo

Arquitectura desplegada: USD $0. Sin tarjeta, sin cargos, dentro del plan.

Lo que se evitó a propósito, con las tarifas reales de `us-east-2`:

Servicio	Coste	Por qué se evita
NAT Gateway	$0,045/h (~$32,85/mes)	factura 24/7 aunque no haya tráfico
Application Load Balancer	$0,0225/h (~$16,43/mes)	ídem
RDS	consumiría ~$75–90 de los $100	factura 24/7; MySQL en contenedor cuesta $0
Dominio propio	~$10–12/año	**fuera de los créditos**: dinero real

Regla operativa: **detener la instancia cuando no se use.** El repositorio trae un script:

```
./controlar-ec2.sh encender   # arranca y espera a que respondan nginx Y el backend
./controlar-ec2.sh apagar     # detiene: deja de consumir horas
./controlar-ec2.sh estado     # estado, IP y URLs
```

Tiempos de un ciclo real: **apagar 37 s, encender 44 s** (la app queda lista ~20 s después de que arranca la instancia). Verificado con un login real por el CDN tras el ciclo.

Para que el stack vuelva solo, `desplegar-ec2.sh` instala la unidad de usuario `neptuno.service` (que ejecuta `podman-compose up -d` al arrancar) y activa `linger`. **No** se usa `podman-restart.service`: solo arranca contenedores con política `always`, y aquí se usa `unless-stopped` para que en local un contenedor que paras a propósito siga parado.

Ojo con una trampa de verificación: `encender` comprueba **nginx y el backend**, no solo `/healthz`. Ese endpoint lo sirve nginx y contesta antes que Spring, así que por sí solo daría un "listo" falso. El backend se confirma con el `401` de `actuator`, que prueba que Spring ya responde.

Con la Elastic IP asociada la dirección sobrevive al Stop/Start, así que el enlace del CDN no se rompe. La instancia detenida conserva el disco de 20 GB.

Guardarraíl adicional: un **presupuesto mensual** con alertas por correo al 50 % y 100 %. Es gratis y avisa antes de que el gasto sorprenda.

---

10. Encendido automático (miércoles antes de clase)

La clase es el miércoles de 10:00 a 14:00 (Colombia). Un cron **dentro** de la instancia no sirve para encenderla: apagada no hay nada que lo ejecute. Tiene que ser desde AWS.

Se usa **EventBridge Scheduler** con un *target universal*, que llama a la API de EC2 directamente, sin Lambda de por medio:

Horario	Expresión	Zona	Acción
`neptuno-encender`	`cron(0 9 ? * WED *)`	`America/Bogota`	`ec2:startInstances`
`neptuno-apagar`	`cron(0 15 ? * WED *)`	`America/Bogota`	`ec2:stopInstances`

Se escribe en hora de Colombia porque EventBridge Scheduler acepta zonas horarias IANA, así que no hay conversiones a mano que equivocar. Ojo: **Colombia es UTC−5 todo el año**, sin horario de verano; la clase de 10:00 COT son las 15:00 UTC.

El rol `neptuno-scheduler` autoriza únicamente `StartInstances` y `StopInstances` **sobre esa instancia concreta**, no `ec2:*`.

Coste: EventBridge Scheduler incluye 14 millones de invocaciones gratuitas al mes y esto son 8 invocaciones mensuales. $0.

Para cambiar la hora, `scheduler update-schedule` con la nueva expresión. Y conviene probar cualquier cambio, porque **una programación que parece correcta puede fallar en silencio** si el rol no se puede asumir, y solo te enterarías el miércoles. La forma de probarlo: crear un horario puntual `at(<fecha-hora>)` a pocos minutos vista, detener la instancia y comprobar que arranca sola.

11. Desmontaje

```
        aws $P ec2 terminate-instances --instance-ids <id>
        aws $P ec2 release-address --allocation-id <eip>      # si no se suelta, se sigue
facturando
        aws $P ec2 delete-security-group --group-id <sg>
        aws $P ec2 delete-key-pair --key-name neptuno
```

Soltar la Elastic IP es el paso que se olvida y sigue generando cargo.

Anexo: por qué no se movió el frontend a S3

La opción canónica sería S3 + CloudFront sirviendo la SPA. Se descartó porque el criterio del curso pide el **frontend en EC2**, y porque añadiría un origen y una sincronización más sin beneficio medible a esta escala. CloudFront delante de la instancia da el mismo resultado —candado, CDN— conservando la arquitectura pedida.